Introduction to Python for Data Analytics

Why Python for Data Analytics

Python is one of the most popular languages in data analytics because it is simple, readable, and extremely powerful for handling data.

Key reasons Python is used in data analytics: - Easy to learn and write - Large ecosystem of analytics libraries - Strong community support - Works well with Excel, databases, and visualization tools

What You Will Learn in This Section

In this section, learners will build a strong Python foundation focused only on what is actually required for data analytics.

You will learn: - How Python fits into the data analytics workflow - Writing and running Python code - Working with numbers, text, and variables - Making decisions using conditions - Repeating tasks using loops - Organizing logic using functions

AI Powered Python for Data Analytics

What Is AI Powered Python

AI powered Python means using artificial intelligence tools to write, understand, debug, and improve Python code faster. Instead of replacing learning, AI acts as a smart assistant that increases productivity.

Why AI Matters for Data Analysts

Modern data analysts do not work alone. AI tools help in: - Generating Python code quickly - Explaining complex logic in simple terms - Debugging errors faster - Improving code quality and efficiency

This reflects how analytics is actually done in real companies today.

How AI Is Used in This Course

In this section, learners will see practical usage of AI while coding in Python.

You will learn: - How to ask AI the right questions for analytics problems - Converting problem statements into Python logic using AI - Understanding AI generated code line by line - Fixing bugs and edge cases with AI assistance

AI as a Learning Accelerator

AI helps beginners overcome fear and confusion by: - Breaking problems into smaller steps - Explaining errors in simple language - Providing multiple approaches to the same problem

Learners still think and decide, AI only speeds up the process.

Installing Python and VS Code

Why Proper Setup Matters

Before writing any code, it is important to set up the right tools. A correct Python and editor setup ensures a smooth learning experience and avoids common beginner issues.

Installing Python

Python is the language we will use for all data analytics tasks.

In this section, you will learn: - How to download and install Python - How to verify that Python is installed correctly - Understanding the Python version and basic setup

Python is available for Windows, macOS, and Linux. Always install the **latest stable version** from the official website.

Installation

1. Open your browser and go to **https://www.python.org**
2. Click on **Downloads** and download the **latest Python version for Windows** (this will download a `.exe` installer).
3. Once the download is complete, **double-click the installer** to start the setup.
4. On the first setup screen, **check the box** that says **"Add Python to PATH"** (this step is very important).
5. Click on **Install Now**.
6. The installer will begin installing Python on your system. This may take a few minutes.

7. When the installation is complete, click **Close**.
8. To make sure everything works correctly, open **Command Prompt** or **Terminal** and run:

```
python --version
```

9. If Python is installed successfully, you will see the installed Python version displayed.

Your Python installation is now complete and ready to use.

Installing Visual Studio Code

Visual Studio Code is a lightweight and powerful code editor widely used by data analysts and developers.

You will learn: - How to install VS Code - Setting up Python support in VS Code - Running Python files inside the editor

Installation

1. Go to **https://code.visualstudio.com**
2. Click **Download**
3. Choose your operating system
4. Download the installer
5. Run the installer
6. Accept the agreement
7. **Check these options**:
    1. Add “Open with Code” action to Windows Explorer file context menu
    2. Add “Open with Code” action to Windows Explorer directory context menu
    3. Register Code as an editor for supported file types
    4. Add to PATH
8. Click **Install**

9. Once the installation is complete, click **Finish** to launch VS Code.

And that’s it! You have both Python and VS Code installed and ready to use.

Installing Python Packages Using Pip

What Are Python Packages

Python packages are reusable pieces of code that add extra functionality to Python. In data analytics, packages help with tasks like data handling, calculations, and visualization.

What Is Pip

Pip is the official package manager for Python. It allows you to download and install Python packages from the Python Package Index.

Installing Packages Using Pip

To install a package, you normally use the pip command in the terminal or command prompt.

Example: pip install package_name

This will download and install the package so it can be used in your Python programs.

If Pip Command Does Not Work

On some systems, the pip command may not work directly due to path or environment issues. In such cases, you can use Python itself to run pip.

Use this command instead:

```
python -m pip install package_name
```

This ensures that pip runs using the same Python version that is installed on your system.

Verifying Installation

After installing a package, you can verify it by importing it in Python or checking the installed packages list.

By the end of this section, learners will be confident in installing and managing Python packages required for data analytics.

Installing Python Packages Using Pip

What Are Python Packages

Python packages are reusable pieces of code that add extra functionality to Python. In data analytics, packages help with tasks like data handling, calculations, and visualization.

What Is Pip

Pip is the official package manager for Python. It allows you to download and install Python packages from the Python Package Index.

Installing Packages Using Pip

To install a package, you normally use the pip command in the terminal or command prompt.

Example: pip install package_name

This will download and install the package so it can be used in your Python programs.

If Pip Command Does Not Work

On some systems, the pip command may not work directly due to path or environment issues. In such cases, you can use Python itself to run pip.

Use this command instead:

```
python -m pip install package_name
```

This ensures that pip runs using the same Python version that is installed on your system.

Verifying Installation

After installing a package, you can verify it by importing it in Python or checking the installed packages list.

By the end of this section, learners will be confident in installing and managing Python packages required for data analytics.

Built-in vs External Packages in Python

In the last lesson, we learned what Python packages are and how to install them using **pip**.
Now, let's understand the difference between **built-in** and **external** packages, and how to use both in real programs.

What Are Python Modules?

A **module** is a single Python file that contains reusable code such as: - Functions - Classes - Variables

Python uses modules to organize code and avoid writing the same logic again and again.

Example:

```
import math
```

Here, `math` is a module that provides mathematical functions.

> A **package** is a collection of multiple related modules.

Built-in (Standard Library) Modules

Built-in modules come pre-installed with Python. You do **not** need to install them using pip.

They are part of Python's **Standard Library** and are available as soon as Python is installed.

Examples of Built-in Modules

- `os` – Interact with the operating system
- `sys` – Work with Python runtime
- `math` – Mathematical functions

External (Third-Party) Packages

External packages are not included with Python by default. They must be installed using **pip**.

These packages are created by the Python community and are widely used in real-world applications.

Examples of External Packages

- `pandas` – Data analysis and manipulation
- `numpy` – Numerical computing
- `matplotlib` – Data visualization

Using an External Package (`pandas`)

Install Pandas

```
pip install pandas
```

If pip does not work:

```
python -m pip install pandas
```

Use Pandas in Python

```
import pandas as pd

data = {
    "Name": ["Alice", "Bob", "Charlie"],
    "Age": [24, 30, 28]
}

df = pd.DataFrame(data)
print(df)
```

Explanation:

- `pandas` is imported using the alias `pd`
- `DataFrame` is used to work with tabular data

Built-in vs External: Key Differences

Feature	Built-in Modules	External Packages
Installation	Comes with Python	Installed using pip
Internet Required	No	Yes (for installation)
Examples	os, math, sys	pandas, numpy, matplotlib
Usage	General-purpose tasks	Specialized tasks (data, ML, web)

Finding More Built-in Modules

Python provides a large number of built-in modules in its standard library. You can explore the complete list here: **https://docs.python.org/3/py-modindex.html**

Variables and Data Types in Python

Before this, we have already installed Python, set up VS Code, and understood how packages work.
Now we start with the core of Python.

This lecture is extremely important because almost everything in data analytics is built on variables and data types.

What is a Variable?

A variable is simply a container that stores a value.

Think of it like a labeled box.
The label is the variable name and the item inside the box is the value.

Example

```
age = 25
```

Here:

- `age` is the variable
- `25` is the value stored inside it

Another example:

```
name = "Harry"
```

Now Python remembers that `name` holds the value `"Harry"`.

In Python, you do not need any special keyword to create a variable. Just write the name, use `=` and assign a value.

What are Data Types?

A data type tells Python what kind of value a variable is holding.

In real life:

- Age is a number
- Name is text
- Price can be a decimal

Python follows the same logic.

Integer (int)

Integers are whole numbers without decimals.

```
students = 100
orders = 45
```

Integers are commonly used for counts and quantities.

Float (decimal numbers)

Floats are numbers that contain decimals.

```
price = 2999.99
rating = 4.5
```

In data analytics, floats are used for prices, averages, percentages, and measurements.

String (text data)

Text values in Python are called strings. Anything written inside single or double quotes is a string.

```
city = "Delhi"
email = "harry@gmail.com"
```

Strings are heavily used for names, cities, emails, and categories.

Boolean (True or False)

Boolean values represent logical conditions.

They can have only two values:

- True
- False

```
is_active = True
is_paid = False
```

Booleans are very common in analytics for flags and conditions.

How Python Identifies Data Types

Python automatically detects the data type of a value.

You do not need to specify it manually.

You can check the data type using the `type()` function:

```
type(age)
type(name)
```

Comments and Type Conversion in Python

In the previous lecture, we learned about variables and data types.
Now we will cover two small but extremely important concepts that you will use all the time in data analytics:

- Comments
- Type conversion

These concepts become even more important when you start working with AI generated code.

Comments in Python

Comments are written for humans, not for Python.

Python completely ignores comments while running the code.
They exist only to explain what the code is doing.

Single Line Comments

In Python, comments start with the `#` symbol.

```
# This stores the user's age
age = 25
```

The line starting with `#` will not be executed.

Why Comments Matter in Data Analytics

In real analytics work:

- Code is read more times than it is written
- You might revisit your code after weeks or months
- AI generated code often needs explanation

Comments help you:

- Understand the logic quickly
- Modify AI generated code safely
- Debug issues faster

Writing comments is a professional habit.

What is Type Conversion?

Type conversion means changing one data type into another.

This is extremely common when working with real data.

Converting String to Integer

Data coming from CSV files or user input is often in string format.

```
age = "25"
```

Even though it looks like a number, it is still a string.

To convert it into an integer:

```
age = int(age)
```

Now you can perform calculations on it.

Converting String to Float

Decimal values are converted using `float()`.

```
price = "1999.99"
price = float(price)
```

This is commonly used for prices, ratings, and averages.

Converting Number to String

Sometimes you need to convert numbers into strings, especially while generating reports or messages.

```
total = 500
message = "Total sales: " + str(total)
```

Common Errors in Type Conversion

Not every string can be converted into a number.

```
value = "abc"
int(value)
```

This will cause an error.

Always validate your data before converting it. Later, we will handle such cases using error handling.

Taking User Input in Python

In many real world programs, data does not always come from files or databases. Sometimes, the user needs to provide input while the program is running.

Python provides a simple way to take input from the user using the `input()` function.

The `input()` Function

The `input()` function pauses the program and waits for the user to type something.

```
name = input()
```

Whatever the user types is stored in the variable `name`.

Taking Input with a Prompt Message

You can guide the user by showing a message inside `input()`.

```
name = input("Enter your name: ")
```

This improves usability and avoids confusion.

Important Note About Input Data Type

By default, **all input taken using `input()` is a string**.

Even if the user enters a number, Python will treat it as text.

```
age = input("Enter your age: ")
type(age)
```

The data type of `age` will be `str`.

Converting User Input to Integer

If you want to perform calculations, you must convert the input.

```
age = int(input("Enter your age: "))
```

Now `age` can be used in numeric operations.

Converting User Input to Float

For decimal values, use `float()`.

```
price = float(input("Enter product price: "))
```

This is common for prices, ratings, and measurements.

Using User Input in Calculations

```
quantity = int(input("Enter quantity: "))
price = float(input("Enter price: "))

total = quantity * price
print("Total amount:", total)
```

Common Input Errors

If the user enters invalid data, the program will crash.

```
age = int(input("Enter age: "))
```

If the user types text instead of a number, an error will occur.

Handling such cases will be covered later using error handling.

Operators in Python

Operators are symbols used to perform operations on values and variables.
They allow you to calculate, compare, assign, and combine data in different ways.

Python provides several types of operators, each serving a specific purpose.

Arithmetic Operators

Arithmetic operators are used to perform mathematical calculations.

Operator	Description	Example
`+`	Addition	`10 + 5`
`-`	Subtraction	`10 - 5`
`*`	Multiplication	`10 * 5`
`/`	Division	`10 / 5`
`//`	Floor Division	`10 // 3`
`%`	Modulus (remainder)	`10 % 3`
`**`	Exponent	`2 ** 3`

Example:

```
a = 10
b = 3

print(a + b)
```

```
print(a // b)
print(a ** b)
```

Assignment Operators

Assignment operators are used to assign values to variables.

Operator	Description	Example
`=`	Assign	`x = 5`
`+=`	Add and assign	`x += 2`
`-=`	Subtract and assign	`x -= 2`
`*=`	Multiply and assign	`x *= 2`
`/=`	Divide and assign	`x /= 2`
`%=`	Modulus and assign	`x %= 2`
`//=`	Floor divide and assign	`x //= 2`
`**=`	Exponent and assign	`x **= 2`

Example:

```
x = 10
x += 5
```

Comparison Operators

Comparison operators compare two values and return either `True` or `False`.

Operator	Description	Example
`==`	Equal to	`a == b`
`!=`	Not equal to	`a != b`
`>`	Greater than	`a > b`
`<`	Less than	`a < b`
`>=`	Greater than or equal	`a >= b`
`<=`	Less than or equal	`a <= b`

Example:

```
a = 10
b = 5

print(a > b)
```

Logical Operators

Logical operators are used to combine conditional statements.

Operator	Description	Example
`and`	Both true	`a > 5 and b < 10`
`or`	Either true	`a > 5 or b > 10`
`not`	Reverse	`not(a > 5)`

Example:

```
a = 10
b = 3
```

```
print(a > 5 and b < 5)
```

Bitwise Operators

Bitwise operators work on binary representations of numbers.

Operator	Description	
`&`	AND	
`|`	OR	
`^`	XOR	
`~`	NOT	
`<<`	Left shift	
`>>`	Right shift	

Example:

```
a = 5
b = 3

print(a & b)
```

Membership Operators

Membership operators are used to test whether a value exists in a sequence.

Operator	Description
`in`	Exists in sequence

Operator	Description
`not in`	Does not exist

Example:

```
numbers = [1, 2, 3]

print(2 in numbers)
```

Identity Operators

Identity operators compare memory locations of objects, not values.

Operator	Description
`is`	Same object
`is not`	Different object

Example:

```
a = [1, 2]
b = a

print(a is b)
```

Operator Precedence

Python evaluates operators in a specific order. Operators with higher precedence are evaluated first.

Precedence Order	Operators	Description
1	`()`	Parentheses
2	`**`	Exponentiation
3	`*` `/` `//` `%`	Multiplication / Division
4	`+` `-`	Addition / Subtraction
5	`==` `!=` `>` `<`	Comparison
6	`not` `and` `or`	Logical operators

Operators at the top are evaluated before operators lower in the table.

Using parentheses is the safest way to control evaluation order and make expressions easier to read.

Strings and String Methods in Python

Strings are used to store text data in Python.
Any value written inside single quotes or double quotes is treated as a string.

Strings are one of the most commonly used data types and Python provides many built in methods to work with them.

Creating Strings

Strings can be created using single quotes or double quotes.

```
name = "Harry"
city = 'Delhi'
```

Both forms work the same way.

Multiline Strings

Multiline strings are created using triple quotes.

```
message = """This is a
multi line
string"""
```

Accessing Characters in a String

Strings are indexed, which means each character has a position.

```
text = "Python"

print(text[0])
print(text[3])
```

Indexing starts from `0`.

String Length

You can find the length of a string using the `len()` function.

```
text = "Python"
length = len(text)
```

String Slicing

Slicing is used to extract a portion of a string.

```
text = "Python"
print(text[0:3])
print(text[2:])
print(text[:4])
```

When slicing a string with negative indexes, Python internally adds the length of the string to the negative value.

```
text[-3:]     # same as text[6-3 : 6]
text[-5:-2]   # same as text[6-5 : 6-2]
```

So Python converts negative indexes like this:

```
-3  ->  6 - 3 = 3
-5  ->  6 - 5 = 1
-2  ->  6 - 2 = 4
```

Common String Methods

Python provides many built in string methods for common operations.

`lower()` and `upper()`

Convert string to lowercase or uppercase.

```
text = "Python"
print(text.lower())
print(text.upper())
```

`strip()`

Removes extra spaces from the beginning and end of a string.

```
text = "  hello  "
print(text.strip())
```

`replace()`

Replaces part of a string with another string.

```
text = "Hello World"
print(text.replace("World", "Python"))
```

`split()`

Splits a string into a list based on a separator.

```
text = "apple,banana,orange"
items = text.split(",")
```

`join()`

Joins elements of a list into a single string.

```
items = ["apple", "banana", "orange"]
text = ",".join(items)
```

`find()`

Finds the position of a substring.

```
text = "Hello World"
position = text.find("World")
```

Returns `-1` if the value is not found.

`startswith()` and `endswith()`

Checks whether a string starts or ends with a given value.

```
email = "test@gmail.com"

print(email.startswith("test"))
print(email.endswith(".com"))
```

String Concatenation

Strings can be combined using the `+` operator.

```
first = "Hello"
second = "World"

result = first + " " + second
```

String Formatting

String formatting allows inserting values into strings.

Using f strings

```
name = "Harry"
age = 25

message = f"My name is {name} and I am {age} years old"
```

Checking String Content

Python provides methods to check string content.

```
text = "Python123"

print(text.isalpha())
print(text.isdigit())
print(text.isalnum())
```

Strings are Immutable

Strings cannot be changed after creation.

```
text = "Python"
text[0] = "J"
```

This will cause an error. Any modification creates a new string.

f-Strings in Python

f-strings (formatted string literals) are an easy and modern way to format strings in Python. They were introduced in **Python 3.6**.

To use an f-string, you add an `f` before the string and place variables or expressions inside `{}`.

Basic Example

```
name = "Harry"
age = 25
```

```
print(f"My name is {name} and I am {age} years old")
```

👉 Here, `{name}` and `{age}` are automatically replaced with their actual values.

Why f-Strings?

- Cleaner than string concatenation (`+`) and `.format()`
- Faster and easier to read
- Supports expressions and method calls

Lists in Python

Lists are used to store multiple values in a single variable.
They are one of the most flexible and commonly used data structures in Python.

A list can store different types of data and can be modified after creation.

Creating a List

Lists are created using square brackets `[]`.

```
numbers = [1, 2, 3, 4]
names = ["Alice", "Bob", "Charlie"]
mixed = [1, "Python", 3.5, True]
```

Accessing List Elements

List elements are accessed using indexes.

```
items = ["apple", "banana", "orange"]

print(items[0])
print(items[2])
```

Indexing starts from `0`.

Negative Indexing

Negative indexes start from the end of the list.

```
items = ["apple", "banana", "orange"]

print(items[-1])
print(items[-2])
```

List Length

Use the `len()` function to get the number of elements in a list.

```
items = ["apple", "banana", "orange"]
count = len(items)
```

List Slicing

Slicing is used to extract part of a list.

```
items = ["apple", "banana", "orange", "mango"]

print(items[1:3])
print(items[:2])
print(items[2:])
```

Modifying List Elements

Lists are mutable, which means their values can be changed.

```
items = ["apple", "banana", "orange"]
items[1] = "grapes"
```

Adding Elements to a List

`append()`

Adds an element to the end of the list.

```
items = ["apple", "banana"]
items.append("orange")
```

`insert()`

Inserts an element at a specific index.

```
items = ["apple", "banana"]
items.insert(1, "orange")
```

`extend()`

Adds multiple elements from another list.

```
items = ["apple", "banana"]
more_items = ["orange", "mango"]

items.extend(more_items)
```

Removing Elements from a List

`remove()`

Removes the first occurrence of a value.

```
items = ["apple", "banana", "orange"]
items.remove("banana")
```

`pop()`

Removes and returns an element at a given index. If no index is provided, it removes the last element.

```
items = ["apple", "banana", "orange"]
items.pop()
items.pop(0)
```

`clear()`

Removes all elements from the list.

```
items = ["apple", "banana"]
items.clear()
```

Finding Elements in a List

`index()`

Returns the index of the first occurrence of a value.

```
items = ["apple", "banana", "orange"]
position = items.index("banana")
```

`count()`

Counts how many times a value appears in the list.

```
items = ["apple", "banana", "apple"]
count = items.count("apple")
```

Sorting Lists

`sort()`

Sorts the list in ascending order.

```
numbers = [3, 1, 4, 2]
numbers.sort()
```

To sort in reverse order:

```
numbers.sort(reverse=True)
```

`sorted()`

Returns a new sorted list without modifying the original list.

```
numbers = [3, 1, 4, 2]
new_list = sorted(numbers)
```

Reversing a List

`reverse()`

Reverses the order of elements in the list.

```
items = ["apple", "banana", "orange"]
items.reverse()
```

Copying Lists

`copy()`

Creates a shallow copy of the list.

```
items = ["apple", "banana"]
new_items = items.copy()
```

Checking Membership

Use the `in` keyword to check if an element exists in a list.

```
items = ["apple", "banana", "orange"]


print("apple" in items)
```

Lists are Mutable

Lists can be modified after creation.

```
items = [1, 2, 3]
items[0] = 10
```

Tuples in Python

Tuples are used to store multiple values in a single variable, similar to lists.
The key difference is that tuples are immutable, which means their values cannot be changed after creation.

Tuples are commonly used when the data should remain constant.

Creating a Tuple

Tuples are created using parentheses `()`.

```
numbers = (1, 2, 3)
names = ("Alice", "Bob", "Charlie")
mixed = (1, "Python", 3.5, True)
```

Single Element Tuple

A tuple with one element must include a trailing comma.

```
single = (5,)
```

Without the comma, Python will treat it as a normal value.

Accessing Tuple Elements

Tuple elements are accessed using indexes.

```
items = ("apple", "banana", "orange")

print(items[0])
print(items[2])
```

Indexing starts from `0`.

Negative Indexing

Negative indexes start from the end of the tuple.

```
items = ("apple", "banana", "orange")

print(items[-1])
print(items[-2])
```

Tuple Length

Use the `len()` function to get the number of elements in a tuple.

```
items = ("apple", "banana", "orange")
count = len(items)
```

Tuple Slicing

Slicing is used to extract part of a tuple.

```
items = ("apple", "banana", "orange", "mango")
```

```
print(items[1:3])
print(items[:2])
print(items[2:])
```

Tuples are Immutable

Tuples cannot be modified after creation.

```
items = ("apple", "banana", "orange")
items[1] = "grapes"
```

This will cause an error.

Tuple Methods

Tuples have fewer built in methods compared to lists.

`count()`

Returns the number of times a value appears in the tuple.

```
items = ("apple", "banana", "apple")
count = items.count("apple")
```

`index()`

Returns the index of the first occurrence of a value.

```
items = ("apple", "banana", "orange")
position = items.index("banana")
```

Looping Through a Tuple

You can loop through tuple elements using a `for` loop.

```
items = ("apple", "banana", "orange")

for item in items:
    print(item)
```

Tuple Packing and Unpacking

Packing

Multiple values can be packed into a tuple.

```
data = 10, 20, 30
```

Unpacking

Tuple values can be unpacked into separate variables.

```
a, b, c = data
```

Converting Between List and Tuple

You can convert a tuple to a list and vice versa.

```
items = ("apple", "banana")
items_list = list(items)


items_tuple = tuple(items_list)
```

When to Use Tuples

- When data should not change
- When you want to protect values from modification
- When returning multiple values from a function

Sets in Python

Sets are used to store multiple values in a single variable.
A set does not allow duplicate values and does not maintain any specific order.

Sets are commonly used when you want unique elements.

Creating a Set

Sets are created using curly braces `{}` or the `set()` function.

```
numbers = {1, 2, 3, 4}
names = {"Alice", "Bob", "Charlie"}
```

To create an empty set, use `set()`.

```
empty_set = set()
```

Using `{}` creates an empty dictionary, not a set.

Unique Elements in a Set

Sets automatically remove duplicate values.

```
numbers = {1, 2, 2, 3, 3, 4}
print(numbers)
```

Accessing Set Elements

Sets do not support indexing or slicing because they are unordered.

To access elements, you must loop through the set.

```
items = {"apple", "banana", "orange"}

for item in items:
    print(item)
```

Adding Elements to a Set

`add()`

Adds a single element to the set.

```
items = {"apple", "banana"}
items.add("orange")
```

`update()`

Adds multiple elements from another iterable.

```
items = {"apple", "banana"}
items.update(["orange", "mango"])
```

Removing Elements from a Set

`remove()`

Removes a specified element. Raises an error if the element does not exist.

```
items = {"apple", "banana"}
items.remove("banana")
```

`discard()`

Removes a specified element without raising an error.

```
items = {"apple", "banana"}
items.discard("grapes")
```

`pop()`

Removes and returns a random element.

```
items = {"apple", "banana", "orange"}
items.pop()
```

`clear()`

Removes all elements from the set.

```
items = {"apple", "banana"}
items.clear()
```

Set Operations

Sets support mathematical set operations.

Union

Combines elements from both sets.

```
a = {1, 2, 3}
b = {3, 4, 5}

result = a.union(b)
```

Intersection

Returns common elements between sets.

```
result = a.intersection(b)
```

Difference

Returns elements present in the first set but not in the second.

```
result = a.difference(b)
```

Symmetric Difference

Returns elements present in either set but not in both.

```
result = a.symmetric_difference(b)
```

Membership Testing

Use the `in` keyword to check if an element exists in a set.

```
items = {"apple", "banana", "orange"}

print("apple" in items)
```

Set Methods Summary

Common set methods include:

- `add()`
- `update()`
- `remove()`
- `discard()`
- `pop()`
- `clear()`
- `union()`
- `intersection()`
- `difference()`
- `symmetric_difference()`

Sets are Mutable

Sets can be modified after creation.

```
items = {"apple", "banana"}
items.add("orange")
```

Important Notes About Sets

- Sets do not maintain order
- Sets do not allow duplicate values
- Sets do not support indexing
- Elements in a set must be immutable

Dictionaries in Python

Dictionaries are used to store data in key value pairs.
Each key is linked to a value, making dictionaries ideal for structured data.

Dictionaries are one of the most important data structures in Python.

Creating a Dictionary

Dictionaries are created using curly braces `{}` with key value pairs separated by colons.

```
student = {
    "name": "Harry",
    "age": 25,
    "city": "Delhi"
}
```

You can also create a dictionary using the `dict()` function.

```
student = dict(name="Harry", age=25, city="Delhi")
```

Accessing Dictionary Values

Values are accessed using keys.

```
print(student["name"])
print(student["age"])
```

Using a key that does not exist will cause an error.

Using `get()`

The `get()` method safely retrieves a value.

```
print(student.get("city"))
print(student.get("email"))
```

If the key does not exist, `None` is returned instead of an error.

Adding and Updating Values

You can add new key value pairs or update existing ones.

```
student["email"] = "test@gmail.com"
student["age"] = 26
```

Removing Dictionary Items

`pop()`

Removes a key and returns its value.

```
age = student.pop("age")
```

`popitem()`

Removes and returns the last inserted key value pair.

```
student.popitem()
```

`del`

Deletes a key value pair.

```
del student["city"]
```

`clear()`

Removes all items from the dictionary.

```
student.clear()
```

Dictionary Keys, Values, and Items

`keys()`

Returns all keys.

```
student.keys()
```

`values()`

Returns all values.

```
student.values()
```

`items()`

Returns key value pairs as tuples.

```
student.items()
```

Looping Through a Dictionary

Loop through keys

```
for key in student:
    print(key)
```

Loop through values

```
for value in student.values():
    print(value)
```

Loop through key value pairs

```
for key, value in student.items():
    print(key, value)
```

Checking for Keys

Use the `in` keyword to check if a key exists.

```
print("name" in student)
```

Copying Dictionaries

`copy()`

Creates a shallow copy of the dictionary.

```
new_student = student.copy()
```

Updating Dictionaries

`update()`

Adds multiple key value pairs at once.

```
student.update({
    "course": "Python",
```

```
    "level": "Beginner"
})
```

Nested Dictionaries

Dictionaries can contain other dictionaries.

```
user = {
    "id": 1,
    "profile": {
        "name": "Harry",
        "email": "test@gmail.com"
    }
}
```

Accessing nested values:

```
print(user["profile"]["email"])
```

Dictionary Characteristics

- Keys must be unique
- Keys must be immutable
- Values can be of any data type
- Dictionaries are mutable

Conditional Statements in Python

Conditional statements are used to make decisions in a program.
They allow the program to execute different blocks of code based on conditions.

Python uses `if`, `elif`, and `else` for conditional logic.

The `if` Statement

The `if` statement checks a condition.
If the condition is true, the code inside the block runs.

```
age = 18

if age >= 18:
    print("Eligible")
```

The `else` Statement

The `else` block runs when the `if` condition is false.

```
age = 16

if age >= 18:
    print("Eligible")
else:
    print("Not eligible")
```

The `elif` Statement

`elif` stands for else if. It is used to check multiple conditions.

```
score = 75

if score >= 90:
    print("Grade A")
elif score >= 60:
    print("Grade B")
else:
    print("Grade C")
```

Indentation in Conditionals

Python uses indentation to define code blocks.

```
if True:
    print("This runs")
    print("This also runs")
```

Incorrect indentation will cause an error.

Comparison Operators in Conditions

Conditions often use comparison operators.

```
x = 10

if x == 10:
    print("Equal")
```

```
if x != 5:
    print("Not equal")
```

Logical Operators in Conditions

Logical operators allow combining multiple conditions.

```
age = 25
country = "India"

if age >= 18 and country == "India":
    print("Allowed")
```

Nested Conditionals

Conditionals can be placed inside other conditionals.

```
age = 20

if age >= 18:
    if age < 60:
        print("Adult")
```

Using `pass` in Conditionals

The `pass` statement is used when a condition is required syntactically but no action is needed.

```
age = 15
```

```
if age >= 18:
    pass
else:
    print("Minor")
```

Conditional Expressions (Ternary Operator)

Python supports one line conditionals.

```
age = 20
status = "Adult" if age >= 18 else "Minor"
```

Common Mistakes

- Forgetting indentation
- Using `=` instead of `==` in conditions
- Writing overly complex nested conditions

Loops in Python

Loops are used to execute a block of code multiple times.
They help reduce repetition and make programs more efficient.

Python mainly provides two types of loops: - `for` loop - `while` loop

---

The `for` Loop

The `for` loop is used to iterate over a sequence.

```
items = ["apple", "banana", "orange"]

for item in items:
    print(item)
```

The loop runs once for each element in the sequence.

Using `range()` with `for` Loop

The `range()` function generates a sequence of numbers.

```
for i in range(5):
    print(i)
```

This will print numbers from `0` to `4`.

`range()` with Start and Step

```
for i in range(1, 10, 2):
    print(i)
```

This prints numbers from `1` to `9` with a step of `2`.

Looping Through a String

Strings can also be iterated character by character.

```
text = "Python"

for char in text:
    print(char)
```

The `while` Loop

The `while` loop runs as long as a condition remains true.

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

Infinite Loop

If the condition never becomes false, the loop runs forever.

```python
while True:
    print("Running")
```

Use infinite loops carefully.

Loop Control Statements

Python provides special statements to control loop execution.

`break`

Stops the loop completely.

```python
for i in range(10):
    if i == 5:
        break
    print(i)
```

`continue`

Skips the current iteration and moves to the next one.

```python
for i in range(5):
    if i == 2:
        continue
    print(i)
```

`pass`

Acts as a placeholder where a statement is required.

```
for i in range(5):
    pass
```

Nested Loops

A loop inside another loop is called a nested loop.

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

Looping with `else`

The `else` block runs when the loop finishes normally.

```
for i in range(3):
    print(i)
else:
    print("Loop finished")
```

The `else` block does not run if the loop is terminated using `break`.

Functions in Python

Functions are used to group reusable code into a single block.
They help make programs organized, readable, and easier to maintain.

Instead of writing the same code again and again, you can define a function and reuse it whenever needed.

Defining a Function

Functions are defined using the `def` keyword.

```
def greet():
    print("Hello")
```

Calling a Function

To execute a function, you call it using its name followed by parentheses.

```
greet()
```

Functions with Parameters

Parameters allow you to pass data into a function.

```
def greet(name):
    print("Hello", name)
```

```
greet("Harry")
```

Here, `name` is a parameter and `"Harry"` is an argument.

Functions with Multiple Parameters

```
def add(a, b):
    print(a + b)

add(5, 3)
```

Return Statement

The `return` statement sends a value back from the function.

```
def add(a, b):
    return a + b

result = add(10, 5)
```

Once `return` is executed, the function stops running.

Default Parameter Values

You can provide default values to parameters.

```
def greet(name="User"):
    print("Hello", name)
```

```
greet()
greet("Harry")
```

Keyword Arguments

Arguments can be passed using parameter names.

```
def user_info(name, age):
    print(name, age)

user_info(age=25, name="Harry")
```

Variable Length Arguments

`*args`

Used to pass multiple positional arguments.

```
def total(*args):
    print(args)

total(1, 2, 3, 4)
```

`**kwargs`

Used to pass multiple keyword arguments.

```
def user_details(**kwargs):
    print(kwargs)
```

```
user_details(name="Harry", age=25)
```

Function Docstrings

Docstrings are used to describe what a function does.

```
def add(a, b):
    """Returns the sum of two numbers"""
    return a + b
```

Scope of Variables

Variables defined inside a function are local to that function.

```
def test():
    x = 10
    print(x)
```

Variables defined outside functions are global.

Function Inside a Function

Functions can be defined inside other functions.

```
def outer():
    def inner():
        print("Inside inner function")
    inner()
```

Lambda Functions

Lambda functions are small anonymous functions written in one line.

```
add = lambda a, b: a + b
print(add(3, 5))
```

Local and Global Variables in Python

Variables in Python have a scope.
Scope defines where a variable can be accessed in the code.

The two most common scopes are local and global.

Local Variables

Local variables are created inside a function.
They can be used only within that function.

```
def show_value():
    x = 10
    print(x)

show_value()
```

Here, `x` is a local variable.

If you try to access it outside the function, it will cause an error.

```
print(x)
```

Global Variables

Global variables are created outside all functions. They can be accessed from anywhere in the program.

```
x = 20

def show_value():
    print(x)

show_value()
```

Here, `x` is a global variable.

Local vs Global Example

```
x = 10

def test():
    x = 5
    print(x)

test()
print(x)
```

Output:

- Inside function: `5`
- Outside function: `10`

The local variable does not affect the global variable.

The `global` Keyword

The `global` keyword is used to modify a global variable inside a function.

```
x = 10
```

```
def update():
    global x
    x = 20

update()
print(x)
```

Without the `global` keyword, Python treats `x` as a local variable.

Why `global` Should Be Used Carefully

Using global variables can make code harder to understand and debug.

It is generally better to:

- Pass values as parameters
- Return values from functions

Global Variables and Function Parameters

A better approach than using `global` is passing data explicitly.

```
def update(x):
    return x + 10

value = 5
value = update(value)
```

Variable Scope Rules

- Local variables exist only inside functions
- Global variables exist throughout the program

- Local variables override global variables with the same name
- The `global` keyword allows modifying global variables

File Handling in Python

File input and output allows a program to read data from files and write data to files.
Python provides built in functions and methods to work with files easily.

Opening a File

Files are opened using the `open()` function.

```
file = open("data.txt", "r")
```

The first argument is the file name. The second argument is the file mode.

File Modes

Commonly used file modes:

Mode	Description
`r`	Read mode
`w`	Write mode
`a`	Append mode
`x`	Create file
`rb`	Read binary
`wb`	Write binary

Reading a File

`read()`

Reads the entire file content as a string.

```
file = open("data.txt", "r")
content = file.read()
print(content)
file.close()
```

`readline()`

Reads one line at a time.

```
file = open("data.txt", "r")
line = file.readline()
print(line)
file.close()
```

`readlines()`

Reads all lines and returns them as a list.

```
file = open("data.txt", "r")
lines = file.readlines()
file.close()
```

Writing to a File

Write Mode (`w`)

Creates a new file or overwrites an existing file.

```
file = open("data.txt", "w")
file.write("Hello Python")
file.close()
```

Append Mode (`a`)

Adds content to the end of a file.

```
file = open("data.txt", "a")
file.write("\nNew line added")
file.close()
```

Using the `with` Statement

The `with` statement automatically closes the file.

```
with open("data.txt", "r") as file:
    content = file.read()
    print(content)
```

This is the recommended way to work with files.

Writing Multiple Lines

```
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]

with open("data.txt", "w") as file:
    file.writelines(lines)
```

Checking File Existence

```
import os

if os.path.exists("data.txt"):
    print("File exists")
```

Reading and Writing Binary Files

```
with open("image.png", "rb") as file:
    data = file.read()
```

```
with open("copy.png", "wb") as file:
    file.write(data)
```

Error Handling in Python

Errors are situations where a program cannot run as expected.
Error handling allows a program to deal with such situations without crashing.

Python uses `try`, `except`, `else`, and `finally` to handle errors.

What is an Error?

An error occurs when Python encounters a problem while executing code.

Common examples: - Dividing by zero - Accessing a file that does not exist - Converting invalid data types

---

The `try` and `except` Block

Code that may cause an error is placed inside the `try` block.
If an error occurs, the code inside `except` runs.

```
try:
    x = int("abc")
except:
    print("An error occurred")
```

The program continues running instead of stopping.

Handling Specific Errors

You can catch specific error types.

```
try:
    x = int("abc")
except ValueError:
    print("Invalid conversion")
```

This helps in handling errors more precisely.

Multiple `except` Blocks

Different errors can be handled separately.

```
try:
    x = int(input("Enter a number: "))
    y = 10 / x
except ValueError:
    print("Please enter a valid number")
except ZeroDivisionError:
    print("Division by zero is not allowed")
```

The `else` Block

The `else` block runs if no error occurs.

```
try:
    x = int("10")
except ValueError:
    print("Error")
```

```
else:
    print("Conversion successful")
```

The `finally` Block

The `finally` block always runs, whether an error occurs or not.

```
try:
    file = open("data.txt", "r")
except FileNotFoundError:
    print("File not found")
finally:
    print("Execution finished")
```

Raising Errors Manually

You can raise an error using the `raise` keyword.

```
age = -5

if age < 0:
    raise ValueError("Age cannot be negative")
```

Using `pass` in Error Handling

The `pass` statement can be used when you want to ignore an error.

```
try:
    x = int("abc")
```

```
except ValueError:
    pass
```

This is generally discouraged unless intentional.

Common Error Types

Some common built in exceptions:

- `ValueError`
- `TypeError`
- `ZeroDivisionError`
- `FileNotFoundError`
- `IndexError`
- `KeyError`

Why Error Handling is Important

- Prevents program crashes
- Makes programs more reliable
- Helps handle unexpected inputs
- Improves user experience

Mini Project: Snake–Water–Gun (Python)

The **Snake–Water–Gun** game is a beginner-friendly project that strengthens logic-building skills and introduces real-world programming flow in Python

Objective

The goal of this mini project is to build a simple command-line game using Python where the user plays **Snake–Water–Gun** against the computer. This project helps reinforce basic Python concepts in a fun and practical way.

Game Rules

- Snake drinks Water → **Snake wins**
- Water disables Gun → **Water wins**
- Gun kills Snake → **Gun wins**
- Same choice by both → **Draw**

Program Explanation

1. The computer randomly selects **Snake**, **Water**, or **Gun** using the `random` module.
2. The user inputs their choice (`s` , `w` , or `g`).
3. A function compares both choices and returns:
    1. `True` → User wins
    2. `False` → User loses
    3. `None` → Draw

4. The result is displayed using **f-strings** for better readability.

Code Implementation

```
import random

def game_win(user, computer):
    if user == computer:
        return None

    # Snake vs Water
    if user == "s" and computer == "w":
        return True
    if user == "w" and computer == "s":
        return False

    # Water vs Gun
    if user == "w" and computer == "g":
        return True
    if user == "g" and computer == "w":
        return False

    # Gun vs Snake
    if user == "g" and computer == "s":
        return True
    if user == "s" and computer == "g":
        return False

rand_no = random.randint(1, 3)

print("Computer's turn: Snake(s), Water (w), Gun (g)")
if rand_no == 1:
    computer = "s"
elif rand_no == 2:
    computer = "w"
else:
    computer = "g"
```

```
user = input("Your turn: Snake(s), Water (w), Gun (g): ").lower()

result = game_win(user, computer) # Returns True if you win, False for lose, None
for draw
print(f"\nYou chose: {user}")
print(f"\nComputer chose: {computer}")

if result is None:
    print("Its a draw!")

elif(result):
    print("You win!")
else:
    print("You lose!")
```

AI Assisted Software Development

AI has changed the way software is written.
Developers no longer rely only on manual coding and documentation.

Modern software development often combines human logic with AI assistance.

ChatGPT and Conversational Coding

ChatGPT allows developers to write code using natural language.

You can: - Ask for code examples - Get explanations for existing code - Debug errors by pasting code and error messages - Convert logic written in English into working code

ChatGPT works best as a thinking and explanation tool.

IDE Integrated AI Development

Tools like GitHub Copilot are directly integrated into code editors.

They: - Suggest code while you type - Complete functions automatically - Understand patterns within your codebase - Speed up repetitive coding tasks

These tools work in real time inside the IDE.

Context Aware Agents and Chat to Code

Modern AI systems can understand project context.

They: - Read multiple files - Understand project structure - Modify existing code safely - Generate code that fits into an ongoing project

Chat to code allows developers to describe changes and let AI implement them across files.

AI Powered Terminals

AI enhanced terminals allow developers to: - Convert plain English commands into terminal commands - Fix command line errors - Explain unfamiliar commands - Automate workflows using natural language

This reduces friction while working with systems and deployments.

Important Note on Using AI for Code

AI can generate code quickly, but it does not guarantee correctness.

Always: - Read the generated code - Understand the logic - Test the output - Modify when needed

AI should assist development, not replace understanding.

Never generate AI code blindly.

ChatGPT Setup and Tips

What is ChatGPT?

ChatGPT is an AI model that can understand natural language and generate human like responses, including code, explanations, and solutions to problems.

ChatGPT vs Other AI Tools

In my personal opinion, ChatGPT is overall better than tools like Gemini and Claude.

Claude Code is very good specifically for coding tasks, but ChatGPT feels more balanced and reliable across coding, explanation, reasoning, and learning use cases.
This comparison is purely my personal opinion based on 4 Years usage experience.

ChatGPT Plans

At the time of writing this, the ChatGPT Go plan is available for free.
This makes it easy for anyone to start using premium features of ChatGPT without any upfront cost.

Generating Python Code with ChatGPT

ChatGPT can generate Python code for a wide range of tasks.

For example, you can ask it to build: - A simple calculator - A GUI based calculator using Tkinter - Small automation scripts - Data processing programs

A basic prompt like:

> Create a calculator GUI using Tkinter in Python

can already generate working code.

Understanding AI Limitations

As your problem becomes more complex, unique, or loosely defined, it becomes harder for AI to generate exactly what you expect.

However, even in such cases, AI often surprises you with: - Well structured code - Logical approaches - Useful starting points

The key is to guide the AI with clear prompts and gradually refine the output.

Final Advice

ChatGPT is a powerful development assistant, not a replacement for thinking.

Always: - Read the generated code - Understand the logic - Modify it based on your requirements

When used correctly, ChatGPT can significantly improve your productivity.

Installing & Using Github Copilot

GitHub Copilot is an AI assistant that works directly inside your code editor.

It provides inline code completions as you type, helping you write code faster with fewer keystrokes.

Copilot can also help fix code by suggesting corrections, completing functions, and improving existing logic.

Like all AI tools, Copilot has limits.
It may generate incorrect or inefficient code, especially for complex or unclear problems.

GitHub Copilot also supports agent based features that can understand project context and help with larger code changes.

Copilot works best when you review, understand, and guide the suggestions instead of accepting them blindly.

Mini Project 2: File Organizer in Python

This project organizes files in a directory into folders based on file extensions using Python.

It scans the current folder, creates category folders like PDFs, Images, and Word files, and moves files into the correct folders automatically.

Code

```
import os
import shutil

FOLDER_PATH = os.getcwd()

FILE_TYPES = {
    "PDFs": [".pdf"],
    "Word_Files": [".docx", ".doc"],
    "Images": [".jpg", ".jpeg", ".png"],
    "Videos": [".mp4", ".mkv"],
    "Text_Files": [".txt"]
}

for folder in FILE_TYPES:
    if not os.path.exists(folder):
        os.mkdir(folder)

for file in os.listdir(FOLDER_PATH):
    if os.path.isdir(file):
        continue

    ext = os.path.splitext(file)[1].lower()

    for folder, extensions in FILE_TYPES.items():
        if ext in extensions:
```

```
            shutil.move(file, os.path.join(folder, file))
            break
```

Mini Project 3: Typing Speed Tester

This project is a command line typing speed tester built using Python.
It displays a random sentence, measures typing time, calculates words per minute, and checks accuracy.

Code

```
import time
import random

sentences = [
    "The quick brown fox jumps over the lazy dog.",
    "A journey of a thousand miles begins with a single step.",
    "This is the way for us to reference the object of the class."
]

def measure_accuracy(user_input, test_sentence):
    correct_chars = sum(1 for a, b in zip(user_input, test_sentence) if a == b)
    accuracy = (correct_chars / len(test_sentence)) * 100 if test_sentence else 0
    return accuracy

def typing_test():
    test_sentence = random.choice(sentences)
    print("Type the following sentence as fast as you can:")
    print(test_sentence)
    input("Press Enter when you are ready...")
    start_time = time.time()
    user_input = input("\nStart typing:\n")
    end_time = time.time()
    time_taken = end_time - start_time
    word_count = len(test_sentence.split(" "))

    print("Results:")
    print(f"Time taken: {time_taken} seconds")
```

```
    print(f"Words typed: {word_count}")
    print(f"Typing speed: {word_count / (time_taken / 60):.2f} words per minute")
    accuracy = measure_accuracy(user_input, test_sentence)
    print(f"Accuracy: {accuracy:.2f}%")

typing_test()
```

Mini Project 4: Quiz App in Python

This project is a simple command line quiz application built using Python.
It displays multiple choice questions, takes user input, checks answers, and shows the final score.

Code

```
def run_quiz():
    questions = [
        {
            "question": "What is the capital of France?",
            "options": ["A) Berlin", "B) Madrid", "C) Paris", "D) Rome"],
            "answer": "C) Paris"
        },
        {
            "question": "What is 2 + 2?",
            "options": ["A) 3", "B) 4", "C) 5", "D) 6"],
            "answer": "B) 4"
        },
        {
            "question": "What is the largest planet in our solar system?",
            "options": ["A) Earth", "B) Jupiter", "C) Saturn", "D) Mars"],
            "answer": "B) Jupiter"
        },
        {
            "question": "Who wrote 'Hamlet'?",
            "options": ["A) Charles Dickens", "B) Mark Twain", "C) William
Shakespeare", "D) Jane Austen"],
            "answer": "C) William Shakespeare"
        }
    ]

    score = 0
```

```
    for index,q in enumerate(questions):
        print(f"Q{index + 1}: {q['question']}")
        for option in q['options']:
            print(option)

        user_answer = input("Your answer(A/B/C/D): ")
        if user_answer.strip().upper() == q['answer'][0]:
            print("Correct!\n")
            score += 1

    print(f"Your final score is {score}/{len(questions)}")

run_quiz()
```

Mini Project 5: PDF Merger GUI in Python

This project is a simple GUI based PDF merger built using Python and Tkinter. It allows users to select multiple PDF files, merge them, and save the output as a single PDF.

Code

```
import tkinter as tk
from tkinter import filedialog, messagebox
from pypdf import PdfWriter
import os

def select_pdfs():
    files = filedialog.askopenfilenames(
        title="Select PDF files to merge",
        filetypes=[("PDF files", "*.pdf")]
    )
    if files:
        pdf_list.delete(0, tk.END)
        for file in files:
            pdf_list.insert(tk.END, file)

def merge_pdfs():
    pdfs = list(pdf_list.get(0, tk.END))
    if not pdfs:
        messagebox.showerror("Error", "No PDF files selected!")
        return

    merger = PdfWriter()
    try:
        for pdf in pdfs:
            merger.append(pdf)
```

```
        output_file = filedialog.asksaveasfilename(
            defaultextension=".pdf",
            filetypes=[("PDF files", "*.pdf")],
            title="Save merged PDF as"
        )
        if output_file:
            merger.write(output_file)
            merger.close()
            messagebox.showinfo(
                "Success",
                f"PDFs merged successfully into {os.path.basename(output_file)}"
            )
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {str(e)}")

root = tk.Tk()
root.title("PDF Merger")
root.geometry("400x300")

select_button = tk.Button(root, text="Select PDF Files", command=select_pdfs)
select_button.pack(pady=10)

pdf_list = tk.Listbox(root, selectmode=tk.MULTIPLE, width=50, height=10)
pdf_list.pack(pady=10)

merge_button = tk.Button(root, text="Merge PDFs", command=merge_pdfs)
merge_button.pack(pady=10)

root.mainloop()
```

Mini Project 6: Water Drinking Reminder with Notifications

This project is a Python based water drinking reminder that sends system notifications at regular intervals using the plyer library.

Code

```
import time
from plyer import notification

def water_reminder(interval_minutes):
    interval_seconds = interval_minutes * 60

    print("Water Drinking Reminder Started")
    print(f"You will get a notification every {interval_minutes} minutes\n")

    while True:
        time.sleep(interval_seconds)
        notification.notify(
            title="Drink Water",
            message="Time to drink water and stay hydrated.",
            timeout=10
        )

water_reminder(30)
```

Mini Project 7: Build a Password Manager Using Python

This project is a simple command line password manager built using Python. It stores passwords in a text file and copies the password to the clipboard when requested without displaying it on screen.

Code

```
import pyperclip
import os

FILE_NAME = "passwords.txt"

def save_password():
    website = input("Enter website: ")
    password = input("Enter password: ")
    with open(FILE_NAME, "a") as f:
        f.write(f"{website}<||>{password}\n")

def get_password():
    website = input("Enter website: ")
    with open(FILE_NAME, "r") as f:
        for line in f:
            if website in line:
                password = line.strip().split("<||>")[1]
                pyperclip.copy(password)
                print("Password copied to clipboard!")
                break
        else:
            print("Website not found.")

def main():
    while True:
```

```
        print("1. Save Password")
        print("2. Get Password")
        print("3. Exit")
        choice = input("Choose an option: ")

        if choice == '1':
            save_password()
        elif choice == '2':
            get_password()
        elif choice == '3':
            print("Exiting...")
            break
        else:
            print("Invalid choice. Please try again.")

main()
```